Design and Implementation of a Web-Based GPS Location Tracking and Management System

Project type: Bachelor of Science (BSc) software engineering thesis

Repository: `MiracleCodez/gps-system`

Implementation status: Working MVP with smartphone geolocation, Express API, SQLite persistence, Leaflet/OpenStreetMap dashboard, tracker registration, trips, and history.

> This document is a thesis-ready technical foundation. Replace bracketed fields such as `[Student Name]`, `[Institution]`, and `[Study Period]` with your official information and adapt the methodology to your department's required format.

Abstract

This project presents the design and implementation of a web-based GPS location tracking and management system that uses a smartphone as the tracking device. The system addresses the cost and accessibility barriers associated with dedicated vehicle tracking hardware by collecting geolocation data through a mobile browser and transmitting it to a web server. Administrators can register trackers, view their latest positions on an interactive map, inspect location history, and manage trips through a browser-based dashboard. The application uses an Express.js backend, a SQLite relational database, and a Leaflet/OpenStreetMap mapping interface. The prototype demonstrates that a low-cost smartphone-based approach can provide the core functions of a fleet or field-activity management system without requiring a physical vehicle or specialized GPS hardware.

Keywords: GPS, location tracking, fleet management, smartphone geolocation, web application, SQLite, Leaflet, geospatial information.

1. Introduction

1.1 Background

Organizations that coordinate vehicles, delivery teams, field workers, school transport, or security personnel need timely information about the location and status of mobile assets. Traditional fleet management systems often depend on dedicated GPS units, vehicle wiring, SIM

subscriptions, and specialized installation. These requirements can be too expensive or impractical for small organizations and student prototypes.

Modern smartphones include GPS receivers, mobile connectivity, browser geolocation APIs, and sufficient processing power to act as lightweight tracking devices. A browser-based solution can therefore collect latitude, longitude, speed, accuracy, and time, then transmit the readings to a central server for monitoring and reporting.

1.2 Problem statement

Small organizations and student project teams may lack access to vehicles, dedicated GPS equipment, and expensive fleet-management platforms. Without a centralized system, location information is difficult to record, view, and analyze. Manual reporting is slow, lacks historical accuracy, and does not provide a shared operational view.

1.3 Aim

To design and implement a web-based GPS location tracking and management system that uses a smartphone as the tracking device and provides a browser-based dashboard for monitoring and managing location data.

1.4 Objectives

1. Analyze the functional and non-functional requirements of a smartphone-based GPS management system.
2. Design a client-server architecture for collecting and storing location data.
3. Implement a smartphone tracking interface that requests geolocation permission and transmits coordinates.
4. Implement an administrative dashboard that displays tracker positions on an interactive map.
5. Store devices, GPS points, and trips in a relational database.
6. Provide location-history and trip-management features.
7. Test the system for functional correctness, data validation, usability, and reliability.

1.5 Research questions

1. How can a smartphone browser be used as a low-cost GPS tracking device?
2. What software architecture is appropriate for collecting, persisting, and visualizing location data?

3. How accurately and reliably can a web application display recent tracker positions?
4. What security, privacy, and connectivity limitations affect a smartphone-based GPS system?

1.6 Scope

The implemented scope includes tracker registration, browser geolocation, location ingestion, latest-location monitoring, route history, trip start/end, device status, API validation, and demo seed data. The project does not include vehicle engine telemetry, fuel sensors, dedicated GPS hardware, payments, emergency dispatch, or production-grade multi-tenant authentication in the current MVP.

1.7 Significance

The project provides a practical and accessible alternative for demonstrations and small deployments. It also gives students a complete software engineering case study involving requirements analysis, REST APIs, database design, geospatial visualization, mobile browser behavior, testing, and deployment considerations.

---

2. Review of the Existing Approach

Existing fleet-management products commonly combine GPS hardware, cellular communication, a cloud backend, and a monitoring dashboard. Their strengths include continuous operation, vehicle integration, and mature alerts. Their weaknesses for a small prototype are hardware cost, installation complexity, and dependence on a vendor platform.

The proposed system uses a smartphone browser instead of dedicated hardware. This reduces the cost of entry and supports rapid demonstration. It also introduces limitations: the user can deny permission, the browser may stop background updates, accuracy depends on device and environment, and connectivity is required for real-time transmission. These trade-offs are explicitly considered in the design.

---

3. Methodology

3.1 Development method

An incremental Agile-inspired approach is appropriate for this project. The system is developed in small increments: requirements and architecture, database/API, dashboard, phone tracker, then testing and documentation. Each increment produces a demonstrable improvement and allows early correction of defects.

3.2 Project phases

Phase	Main activities	Deliverable
Requirements	Identify actors, use cases, constraints, and success criteria	Software requirements specification
Analysis and design	Select architecture, data model, interfaces, and security boundaries	UML/DFD/ERD diagrams
Implementation	Build API, database, dashboard, tracker page, and map integration	Working MVP
Testing	Execute functional, validation, usability, and integration tests	Test results and defect log
Evaluation	Compare objectives with observed behavior and limitations	Evaluation chapter
Documentation	Prepare diagrams, screenshots, user guide, and maintenance notes	Thesis and appendices

3.3 Technology selection

Layer	Technology	Reason for selection
Client	HTML, CSS, JavaScript	Low barrier, browser-native, easy to demonstrate
Geolocation	Browser Geolocation API	Uses the smartphone’s built-in location capability
Server	Node.js and Express.js	Lightweight REST API and static-file serving
Database	SQLite	Relational persistence with minimal setup for a BSc prototype
Map	Leaflet.js and OpenStreetMap	Interactive map with open and accessible tooling
Version control	Git and GitHub	Traceable development history and backup

4. Requirements Analysis

4.1 Actors

- **Administrator/Fleet manager:** registers trackers, views the map, inspects telemetry and history, and reviews trips.
- **Phone tracker/field user:** grants location permission, starts or stops tracking, starts or ends a trip, and transmits location data.
- **System:** validates incoming data, stores records, computes online/offline status, and returns information to clients.

4.2 Functional requirements

ID	Requirement	Priority
FR-01	The system shall list registered trackers.	Must
FR-02	The system shall allow an administrator to register a tracker name and type.	Must
FR-03	The tracker page shall request permission before accessing location.	Must
FR-04	The tracker shall send latitude, longitude, speed, accuracy, and timestamp to the API.	Must
FR-05	The system shall validate coordinate ranges and tracker identity.	Must
FR-06	The dashboard shall display the latest location of each tracker on a map.	Must
FR-07	The system shall persist location points for later review.	Must
FR-08	The system shall display location history for a selected tracker.	Must
FR-09	The tracker shall start and end trips.	Must
FR-10	The system shall show online/offline status based on recent location activity.	Should
FR-11	The system shall provide a health endpoint for operational checks.	Should
FR-12	The system should support demo data when no physical phone is connected.	Should

4.3 Non-functional requirements

ID	Requirement
NFR-01	The interface should be responsive on laptop and smartphone screens.
NFR-02	The API should reject malformed or out-of-range location data.
NFR-03	The dashboard should refresh recent tracker information automatically.
NFR-04	The system should preserve records across server restarts using SQLite.
NFR-05	The UI should communicate connection, permission, and validation errors clearly.
NFR-06	The system should use HTTPS in real deployment because browser geolocation requires a secure context.
NFR-07	Location data should be treated as sensitive personal information and access should be restricted in production.

4.4 Constraints and assumptions

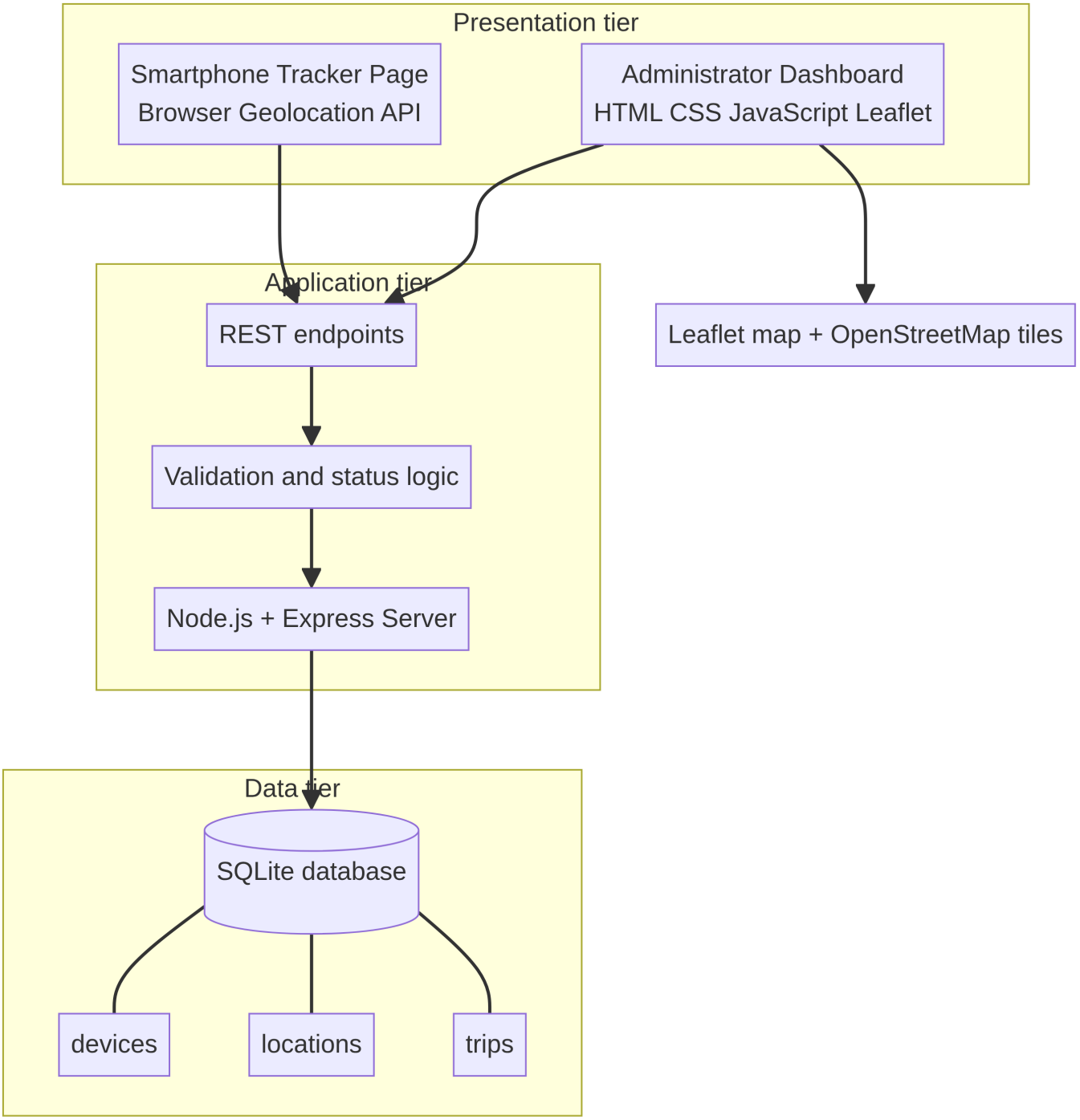
The prototype assumes a smartphone with location services, a browser that supports `navigator.geolocation`, and network access to the API. A phone may be used while walking to simulate a vehicle route. For a real deployment, the application should use authenticated accounts, HTTPS, a production database, retention policies, and explicit user consent.

---

5. System Design

5.1 High-level architecture

The system follows a three-tier architecture. The presentation tier contains the dashboard and tracker pages. The application tier contains Express routes and validation logic. The data tier contains SQLite tables for devices, locations, and trips. Leaflet renders map tiles and markers in the browser; the application itself stores only tracker telemetry.



5.2 Operational workflow

1. An administrator opens the dashboard and registers or selects a tracker.
2. A phone user opens the tracker page and grants geolocation permission.
3. The browser's geolocation watcher receives coordinates.
4. The tracker page sends a JSON request to `POST /api/locations`.
5. The server validates the device and coordinate ranges.

6. SQLite stores the location and updates the device's activity status.
7. The dashboard refreshes and displays the latest point and route history.
8. A trip record is created when the user starts a trip and completed when the user ends it.

5.3 Security and privacy design

The MVP validates input, constrains JSON payload size, uses parameterized SQLite statements, and does not expose database files through the static directory. For thesis evaluation, the production hardening plan should add HTTPS, password hashing, session or token authentication, role-based authorization, audit logs, rate limiting, CSRF protection where applicable, and configurable location-retention/deletion policies.

---

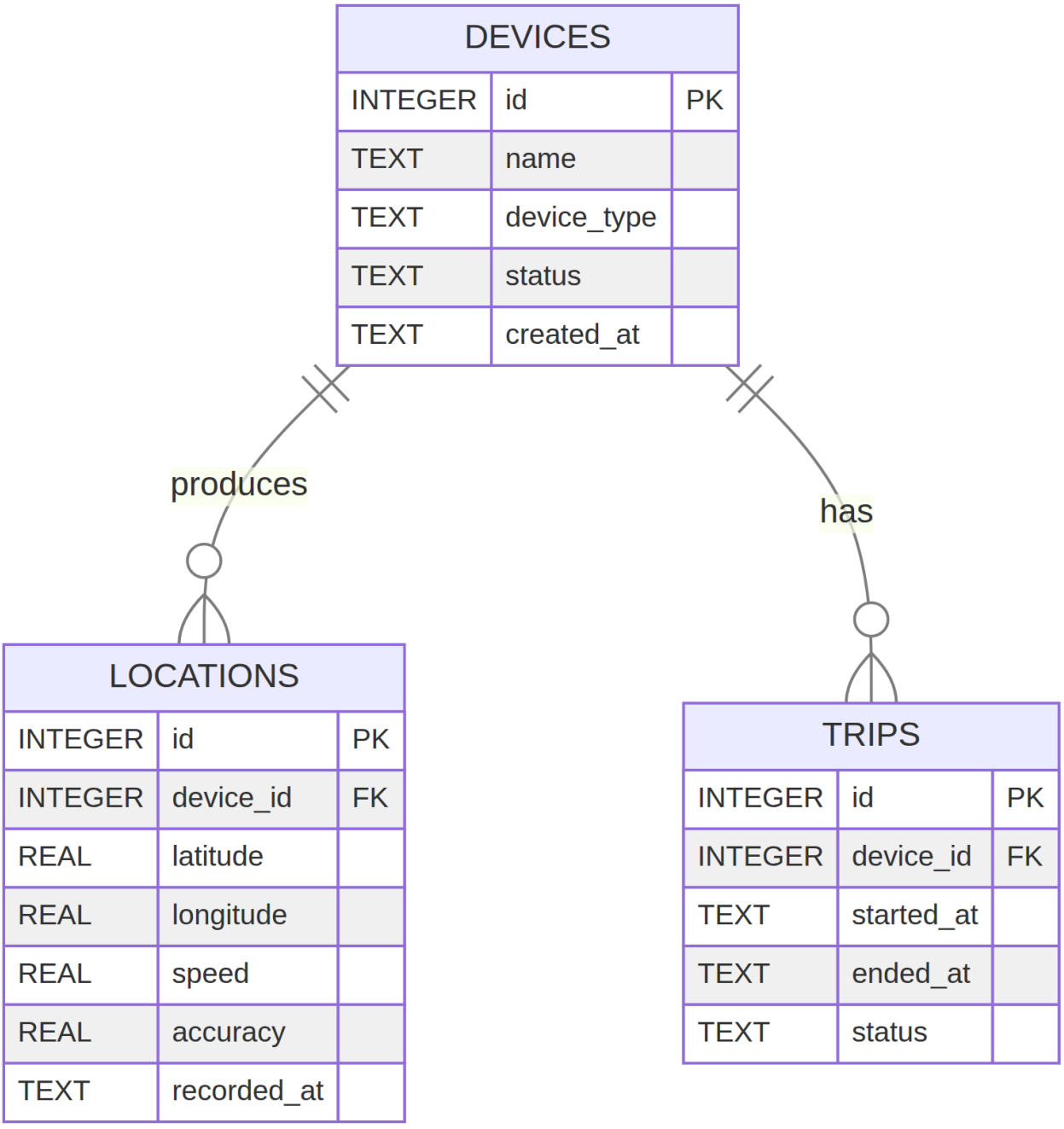
6. Database Design

6.1 Entities

- **Device:** a registered smartphone or GPS unit that can submit location data.
- **Location:** one geospatial observation belonging to a device.
- **Trip:** a period of tracking activity associated with a device.

6.2 Relationships

One device can produce many locations. One device can have many trips. Each location and trip belongs to exactly one device. The `device_id` foreign key enforces these relationships.


DEVICES
INTEGER id PK
TEXT name
TEXT device_type
TEXT status
TEXT created_at
produces
has
LOCATIONS
INTEGER id PK
INTEGER device_id FK
REAL latitude
REAL longitude
REAL speed
REAL accuracy
TEXT recorded_at
TRIPS
INTEGER id PK
INTEGER device_id FK
TEXT started_at
TEXT ended_at
TEXT status

6.3 Relational schema

```
DEVICES(
  id PK,
  name,
  device_type,
  status,
  created_at
)

LOCATIONS(
  id PK,
  device_id FK → DEVICES.id,
  latitude,
  longitude,
  speed,
  accuracy,
  recorded_at
)

TRIPS(
  id PK,
  device_id FK → DEVICES.id,
  started_at,
  ended_at,
  status
)
```

6.4 Data dictionary

Table	Field	Type	Key	Description
devices	id	INTEGER	PK	Unique tracker identifier
devices	name	TEXT	—	Human-readable tracker name
devices	device_type	TEXT	—	Smartphone or GPS device
devices	status	TEXT	—	Current derived status: online/offline
devices	created_at	TEXT	—	Registration time
locations	id	INTEGER	PK	Unique location record
locations	device_id	INTEGER	FK	Device that produced the point
locations	latitude	REAL	—	Latitude from -90 to 90
locations	longitude	REAL	—	Longitude from -180 to 180
locations	speed	REAL	—	Speed in km/h; defaults to zero
locations	accuracy	REAL	—	Approximate accuracy in metres
locations	recorded_at	TEXT	—	ISO timestamp of observation
trips	id	INTEGER	PK	Unique trip identifier
trips	device_id	INTEGER	FK	Tracker assigned to the trip
trips	started_at	TEXT	—	Trip start time
trips	ended_at	TEXT	—	Trip completion time
trips	status	TEXT	—	Active or completed

7. Implementation Description

7.1 Backend

`server.js` initializes SQLite tables on startup, seeds a demo tracker when the database is empty, serves static files, and exposes REST endpoints. The location endpoint checks that the device exists and that latitude and longitude are within valid geographic ranges before inserting a record. The dashboard obtains the latest point using a descending timestamp query.

7.2 Tracker client

`tracker.html` provides a phone-sized interface. `tracker.js` calls `navigator.geolocation.watchPosition` with high-accuracy options. Each successful reading is converted into JSON and posted to the server. Permission failures, unavailable location services, and API failures are shown to the user.

7.3 Dashboard client

`index.html` and `app.js` provide the operations dashboard. The client initializes a Leaflet map, requests `/api/devices`, creates markers for trackers with a location, and refreshes every ten seconds. Selecting a device requests its location history and draws a route polyline.

7.4 API specification

Method	Endpoint	Request body	Response
GET	`/api/health`	—	Service health and timestamp
GET	`/api/devices`	—	Devices with latest location
POST	`/api/devices`	`{ name, deviceType }`	Created device
GET	`/api/devices/:id/locations`	`?limit=100`	Ordered location points
POST	`/api/locations`	`{ deviceId, latitude, longitude, speed, accuracy }`	Created point
POST	`/api/trips/start`	`{ deviceId }`	Active trip
POST	`/api/trips/end`	`{ deviceId }`	Completion result
GET	`/api/trips`	—	Recent trips

8. Testing and Evaluation Plan

8.1 Test levels

- **Unit-level validation:** JavaScript syntax checks and API input validation.
- **Integration testing:** confirm that tracker requests create database records and appear on the dashboard.
- **System testing:** exercise the full phone-to-server-to-map workflow.
- **Usability testing:** ask representative users to register a tracker, start tracking, and find route history.

- **Security testing:** verify malformed coordinates, unknown devices, oversized payloads, and unauthorized access once authentication is added.

8.2 Functional test cases

ID	Test	Expected result
TC-01	Open `/api/health`	Returns HTTP 200 and `ok: true`.
TC-02	List devices	Returns the seeded or registered tracker.
TC-03	Register a tracker without a name	Returns HTTP 400 and a useful error.
TC-04	Submit valid coordinates	Returns HTTP 201 and stores one location.
TC-05	Submit latitude above 90	Returns HTTP 400 and stores nothing.
TC-06	Submit an unknown device ID	Returns HTTP 404.
TC-07	Start and end a trip	Creates an active trip then marks it completed.
TC-08	Open dashboard	Returns HTTP 200 and loads the map shell.
TC-09	Open phone tracker	Returns HTTP 200 and loads tracker controls.
TC-10	Select a tracker	Displays telemetry, point count, table, and route when history exists.
TC-11	Deny browser geolocation	Shows a permission error and stops tracking.
TC-12	Restart the server	Existing SQLite records remain available.

8.3 Evaluation metrics

Suggested measurements include API response time, location-ingestion success rate, time from phone update to dashboard refresh, coordinate accuracy reported by the device, and task-completion rate during usability testing. Record the browser, phone model, operating system, network type, test route, and number of updates for reproducibility.

---

9. Limitations and Future Work

The browser tracker may stop when the phone sleeps or the browser is backgrounded. GPS accuracy can fall indoors or near tall buildings. The current prototype has no user authentication, so it should not be exposed as a production fleet system without adding access control. SQLite is appropriate for a single-node prototype but should be replaced with PostgreSQL/PostGIS for concurrent users and advanced geospatial queries.

Future work can add geofences, route-deviation alerts, push notifications, offline buffering, a progressive web app, background location support where permitted, role-based accounts, PostgreSQL/PostGIS, analytics, and a dedicated hardware adapter.

---

10. Recommended Thesis Chapter Structure

1. **Chapter One — Introduction:** background, problem, aim, objectives, questions, scope, significance.
2. **Chapter Two — Literature Review:** GPS tracking, smartphone geolocation, web fleet systems, mapping APIs, privacy, and related systems.
3. **Chapter Three — Methodology and System Analysis:** development method, requirements, feasibility, actors, use cases, and constraints.
4. **Chapter Four — System Design and Implementation:** architecture, database, API, interface design, algorithms, and implementation screenshots.
5. **Chapter Five — Testing, Results, and Discussion:** test cases, measurements, observations, limitations, and comparison with objectives.
6. **Chapter Six — Summary, Conclusion, and Recommendations:** contribution, conclusion, and future work.
7. **Appendices:** source-code excerpts, API requests, database schema, test evidence, user guide, and diagrams.

Appendix A — User Guide

Administrator

Open the dashboard, select the plus button in the Trackers panel, enter a tracker name, and choose a device type. Select a tracker to view its latest coordinates and history. The map refreshes automatically every ten seconds.

Phone user

Open `/tracker.html` on the smartphone, press **Start tracking**, grant location permission, and optionally press **Start trip**. Keep the page open while moving. Press **Stop tracking** when finished and **End trip** to complete the trip record.

Demonstration without a car

Use the smartphone while walking on campus or around the neighborhood. The phone acts as the GPS device. The dashboard displays the route as if the phone were installed in a vehicle. The seeded demo tracker can be used when a phone is unavailable.

Appendix B — Reproducibility Commands

```
npm install
npm start

# Health check
curl http://localhost:3000/api/health

# List trackers
curl http://localhost:3000/api/devices

# Submit a sample point
curl -X POST http://localhost:3000/api/locations \
  -H 'Content-Type: application/json' \
  -d '{"deviceId":1,"latitude":6.528,"longitude":3.383,"speed":12,"accura
```